Research Notes: CI/CD & GitLab Runners

Use these notes to prepare for your project transition. Focus on understanding the *concept* rather than memorizing syntax.

1. GitLab CI/CD: The "Execution Engine"

CI/CD automates the running of your code. Instead of manually running `python main.py`, you let GitLab do it.

- **Concept:** Every time code is pushed or an issue is updated, GitLab looks for a `.gitlab-ci.yml` file in your repository.
- **Jobs:** A "job" is the smallest unit of work. One job = your agent running one task (e.g., "Analyze Issue #5").
- **Why:** It ensures the agent runs in a clean, consistent environment every time.
- **Read:** Pipeline Architecture

2. GitLab Runner: The "Worker"

The "Runner" is the program that actually executes the job on a machine.

- **Concept:** Think of the GitLab server as the "manager" and the Runner as the "employee." The Runner stays on your PC, listens for tasks, and executes them when asked.
- **Registration:** You install the software on your PC and use a "token" to connect it to your GitLab project/group.
- **Read:** GitLab Runner Overview

3. Designing for "General" Use (Important!)

To make the agent "general," it must not care *which* repository it is working on.

- **Avoid hardcoding:** Never write `project_id = 123`.

- **Use Predefined Variables:** GitLab provides variables like `$CI_PROJECT_ID`, `$CI_COMMIT_REF_NAME`, and `$CI_JOB_TOKEN`.
- **Your Task:** Refactor your agent to read these variables from the system environment instead of your code.
- **Read:** Predefined Variables Reference

4. Action Items for You

- **Step 1:** Read the links above. Do not try to build yet—just understand the flow.
- **Step 2:** Install GitLab Runner on your PC (follow the standard OS guide).
- **Step 3:** Register the Runner to your project so you can see it show up in the GitLab "Runners" settings page.